Centralized Intrusion Detection on CICIoT2023

Research assessment and next experimental plan: MLP, Random Forest, and XGBoost

Research report | 12 October 2026 | Focus: accuracy and macro F1

Research objective. We are investigating whether tree-based models can improve the 34-class CICIoT2023 intrusion-detection results obtained by our centralized MLP, and which improvements arise from model choice versus data preparation, class balancing, and training configuration.

The published results reviewed here provide relevant comparison targets, not predictions for our forthcoming Kaggle experiments. All new measurements will use documented data partitions and explicitly reported class definitions.

1. Where our research currently stands

Our prior controlled study compared centralized MLP training against three federated configurations on the same cleaned CICIoT2023 data. The results below are the recorded test results, not new measurements.

Experiment	Accuracy	Macro F1
Centralized MLP	77.93%	56.70%
FedAvg — IID	77.16%	52.90%
FedAvg — non-IID	65.74%	28.80%
FedProx — non-IID	64.91%	23.36%

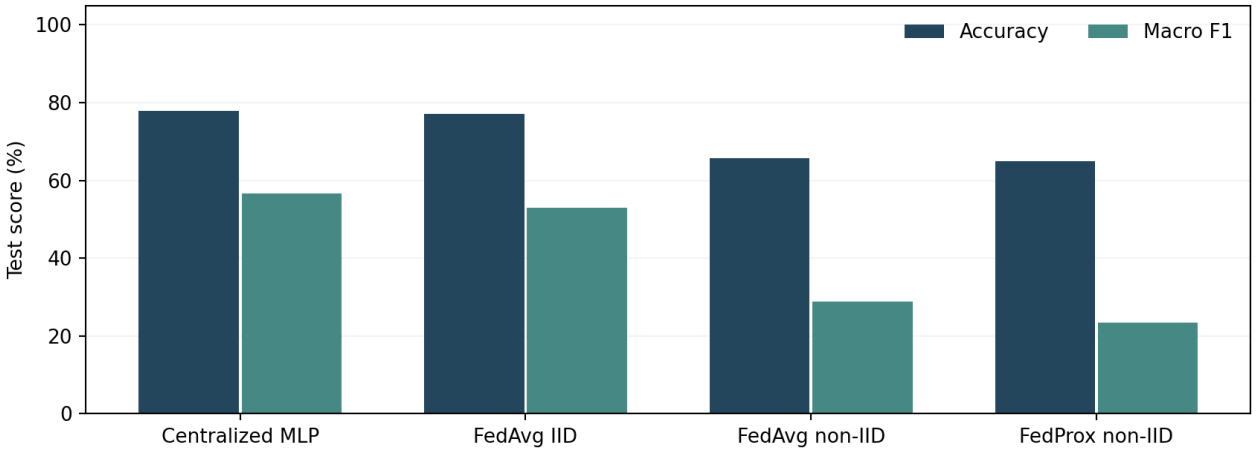


Figure 1. Recorded accuracy and macro F1 in the preceding research report.

- Cleaned dataset: 45,018,243 samples; stratified 80/20 split: 36,014,594 training and 9,003,649 testing samples.
- Prediction task: 34 fine-grained classes (33 attacks and BENIGN) with 39 numerical features.
- Centralized architecture: MLP 39–128–64–34; Adam optimizer; learning rate 0.001; batch size 8192; only three epochs.
- Class weighting was disabled. The largest class contained 6,893,196 samples while the smallest contained only 1,196.

- The centralized MLP produced zero F1 for five classes, showing that minority-class problems existed before federation.

Interpretation. Our 77.93% accuracy and 56.70% macro F1 describe one short-training MLP configuration; they do not establish the best achievable neural-network performance. The strongest observed degradation came from severe non-IID client label skew.

2. Are Random Forest and XGBoost actually better?

Random Forest and XGBoost are plausible alternatives for numerical network-flow features because they can represent nonlinear, threshold-like decision boundaries without requiring the same gradient-based neural-network optimization. Neither is universally superior; we will test their performance directly.

Model mechanisms

- Random Forest trains multiple trees using randomized observations and feature subsets; predictions are combined by voting. Its main advantage is a relatively robust tabular baseline.
- XGBoost builds boosted trees sequentially, using gradients of the loss to improve the ensemble. It supports GPU histogram training and flexible weighting.
- Our MLP learns numerical weight matrices using backpropagation and gradient-based optimization. More training epochs and appropriately weighted losses may substantially change its results.

Aspect	MLP	Random Forest	XGBoost
Method	Gradient-based network	Bagged trees	Boosted trees
Scaling	Usually helpful	Not required	Not required
Nonlinear learning	Yes	Yes	Yes
Imbalance tools	Weighted loss / sampling	Class weighting / sampling	Sample weighting / sampling
GPU path	Supported	Usually CPU in sklearn	Supported

3. What published research actually demonstrates

The following sources are linked to their original publisher pages. We distinguish their task granularity, evaluation setting, and preprocessing to avoid interpreting unrelated headline scores as direct model comparisons.

Study A — Optimized gradient boosting framework (Scientific Reports, 2026)

Open original paper — Scientific Reports

Setting	Reported result
Task	34 fine-grained CICIoT2023 classes
Model	XGBoost
Test accuracy	99.48%
Test macro F1	88.76%

Setting	Reported result
Pipeline	Stratified sampling, hyperparameter search, feature selection and cross-validation

This paper offers a relevant high-performance 34-class comparison target. Importantly, its 46-feature to 23-feature ablation and preprocessing differ from our previous 39-feature baseline. These 99.48% and 88.76% scores cannot be attributed exclusively to replacing an MLP with XGBoost. The paper also identifies weaker performance in low-support attack classes.

Study B — Family-aware hierarchical XGBoost (CMES, 2026)

Open original paper — Computer Modeling in Engineering & Sciences

Setting	Reported result
Task	Attack-family classification, not 34 fine-grained labels
Evaluation subset	30% stratified CICIoT2023 development split
Hierarchical XGBoost macro F1	83.80% (five-fold mean)
Hierarchical XGBoost accuracy	99.42%
Flat Random Forest macro F1	84.23% (five-fold mean)

The paper is informative about hierarchy, minority attack families, and tree-model comparison; its family-label results are not directly comparable to our 34-class macro F1. The previously circulated 85.85% and 79.28% figures were not confirmed in the published main comparison and are excluded from this verified report.

Study C — Intrusion detection using machine-learning methods (IET Smart Cities, 2025)

Open original paper — IET Smart Cities

34-class model	Accuracy	Reported F1
Random Forest	88.39%	57.73%
Weighted Random Forest	89.12%	60.18%
XGBoost	19.11%	3.43%

The article reports that XGBoost under its configuration substantially underperformed Random Forest. Its F1 metric appears in a wider class-comparison table; metric definitions and experimental design must be checked before treating it as fully equivalent to our macro F1. Together, the literature demonstrates strong sensitivity to data processing and training settings.

How we interpret the research comparisons

Result	Accuracy	Macro F1 / F1	Comparability
Our centralized MLP (34 classes)	77.93%	56.70% macro F1	Internal baseline
Scientific Reports XGBoost (34 classes)	99.48%	88.76% macro F1	Same label count; different pipeline
CMES hierarchical XGBoost (attack families)	99.42%	83.80% macro F1	Different label granularity
CMES flat Random Forest (attack families)	—	84.23% macro F1	Different label granularity
IET standard Random Forest (34 classes)	88.39%	57.73% reported F1	Different pipeline; verify metric protocol

Comparison principle. We will not claim to reproduce a published result solely because accuracy is similar. Meaningful comparisons require aligned class labels, dataset versions, sampling strategies, feature sets, held-out test methodology, and macro-F1 definitions.

4. Kaggle dataset comparison and provenance

- Previously used data mirror — mashrurfaiyaz28/ciot2023-2
- Teammate-provided data mirror — himadri07/ciciot2023

Our original preparation notebook loaded 63 CSV shards from the first mirror and retained 45,018,243 cleaned rows. The newer Kaggle listing reports a roughly 501 MB dataset. The new mirror's complete row count, schema, class distribution, duplicate rate and sample selection policy have not yet been established from the public listing.

- We will verify feature names, types and order, including whether both mirrors represent the same set of numerical traffic attributes.
- We will check all 34 expected labels, their normalized spellings, and minority-class support.
- We will investigate duplicate records, null and infinite values, train/test leakage, and the sampling or balancing method used in each mirror.
- We will retain our original full-scale dataset for the primary controlled experiment until equivalence of the new mirror is demonstrated.

5. What we are planning to do next

Overall experimental sequence

We will first compare baseline model families under one controlled protocol. We will then isolate class-imbalance interventions and finally tune the strongest candidates before returning to full-scale training.

Stage	Models / treatment	Primary research question
Stage 1	Original MLP; Random Forest; XGBoost; longer-trained MLP	Does model family or training duration explain the score gap?
Stage 2	Class weights; majority undersampling; combined treatment	How much can minority-class macro F1 improve?

Stage	Models / treatment	Primary research question
Stage 3	Model-specific tuning and full-data evaluation	Do improvements persist at full scale?

Stage 1 — Controlled model-family comparison

ID	Experiment	Purpose
E1	Reproduced original MLP	Re-establish the 77.93 / 56.70 baseline
E2	Basic Random Forest	Measure the unbalanced RF baseline
E3	Basic XGBoost	Measure the unbalanced boosting baseline
E4	Longer-trained MLP	Isolate the three-epoch training-budget limitation

For Kaggle development, we will first select a fixed stratified 3–5 million-row subset from the training partition, retaining all classes. We will preserve a separate validation partition and will not inspect the test labels while tuning. The original notebook's CSV-file fraction approach will be replaced with row-level stratified sampling because choosing files can unintentionally change class composition.

Stage 2 — Class-imbalance experiments

ID	Treatment	Research purpose
E5	Original unbalanced training	Control condition
E6	Class/sample weighting	Increase minority contribution to optimization
E7	Majority-class undersampling	Reduce dominant-class influence
E8	Moderate undersampling plus weights	Test a combined intervention

In the earlier cleaned data, DDOS-ICMP_FLOOD contributed 6,893,196 samples while UPLOADING_ATTACK contributed 1,196. We will compare class weights and controlled resampling without changing validation or test distributions. For Random Forest we will test balanced_subsample; for XGBoost we will use row-level sample_weight; for MLP we will test weighted cross-entropy. Weight magnitude and sampling ratio will be validation-tuned rather than assumed.

Stage 3 — Hyperparameter optimization

Model	Parameters selected for controlled sweeps
Random Forest	n_estimators, max_depth, min_samples_leaf, max_features
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight
MLP	epochs, learning rate, batch size, hidden dimensions, class weights

Our initial XGBoost configuration will use the multi:softprob objective, 34 output classes, tree_method=hist, device=cuda, 300 estimators, maximum depth 6, learning rate 0.1, subsample 0.8, and colsample_bytree 0.8. This is a starting point and not an already optimized setting. We will measure Kaggle GPU memory and RAM

consumption and use validation-based early stopping where supported.

Proposed performance milestones

Milestone	Accuracy	Macro F1
Previous centralized MLP (measured)	77.93%	56.70%
Initial improvement (target)	90%	70%
Strong improvement (target)	95%	80%
Published-result stretch target	~99%	~89%

The three improvement rows are planning milestones, not forecast outcomes. We will prioritize macro F1 during model selection and report accuracy alongside it. For every configuration we will also retain class-level F1, normalized confusion matrices, runtime, feature configuration, and a reproducible random seed.

6. How the new centralized results connect to federated learning

We have already measured centralized MLP, FedAvg-IID, FedAvg-non-IID, and FedProx-non-IID. The next centralized study will determine whether the main limitations are model-specific, training-budget-related, or rooted in imbalance. Only after these effects are separated will we expand the federated experiments.

- Phase A — Existing centralized MLP and federated baselines: completed.
- Phase B — Controlled centralized MLP, Random Forest and XGBoost comparison: planned.
- Phase C — Transfer validated MLP-side improvements (for example weighting and longer training) to FedAvg and FedProx: planned.
- Phase D — Investigate federated tree-specific algorithms separately; standard neural-network FedAvg parameter averaging does not directly apply to tree ensembles.
- Phase E — Repeat key experiments under multiple random seeds and evaluate performance under IID and severe non-IID partitions.

We will separately investigate leakage-resistant evaluation and, where metadata permit, group- or capture-aware holdouts. Such splits assess generalization more rigorously than randomized row splits and may produce lower but more meaningful scores.

7. Immediate Kaggle execution plan

- Audit both Kaggle mirrors and retain a versioned manifest of columns, labels, counts, cleaning steps, and partition hashes.
- Create one fixed train/validation/test protocol, then a stratified development subset from training data only.
- Run E1–E3 first on identical data without class balancing; add E4 to determine the effect of more neural-network training.
- Save predictions, confusion matrices, class-level classification reports, accuracy, macro F1, timing, and complete hyperparameters.
- Select the strongest two candidates for E5–E8 balancing experiments.
- Tune final candidates using validation macro F1; then test once on the held-out test partition and repeat important experiments over at least three seeds.

Next experimental checkpoint. Our first deliverable will be a four-row controlled E1–E4 comparison table generated from Kaggle notebook outputs. That result will establish whether XGBoost and Random Forest improve our specific 34-class task before the balancing and full-scale studies begin.

8. Sources and research materials

- Our original dataset mirror
- Our teammate's proposed dataset mirror
- Scientific Reports paper — DOI 10.1038/s41598-026-47399-5
- CMES paper — DOI 10.32604/cmes.2026.085678
- IET Smart Cities paper — DOI 10.1049/smc2.70014

Internal source: Federated Learning for IoT Intrusion Detection — A Controlled Evaluation of Centralized Training, FedAvg, and FedProx under IID and Non-IID Data (project research report, October 2026), together with the dataset-preparation and centralized-baseline notebooks.